

HW 3: Priority-based Scheduler for xv6

Task 1. Modify the provided ps command to print the priority of each process.

I added a new priority field to the pstat structure in kernel/pstat.h so that the ps command would show the priority of each process. I copied the priority of each process (p->priority) into this new variable from the procinfo() method. No other modifications were required there because procinfo() is already used by sys_getprocs() to fill in process information. Finally, I added and printed the priority column by changing user/ps.c. Following task 1's execution and rebuild, the ps command accurately showed all processes along with their priorities.

Task 2. Implement a priority-based scheduler.

For Task 2, I modified param.h, proc.h, and proc.c to implement a priority-based scheduler. To choose between the round-robin and priority scheduling policies, I added constants to param.h. To store the priority value for each process, I introduced a priority field to the proc structure in proc.h. Instead of employing round-robin order, I modified scheduler() in proc.c to choose the process with the highest priority. Additionally, I set allocproc()'s default priorities. The output demonstrated that higher-priority processes (A and C, with priority 40) executed before lower-priority ones (B, with priority 39) upon rebuilding and execution of the given task2.c test, validating proper behavior. This job taught me how to include adjustable scheduling algorithms into an OS kernel and how scheduling settings impact CPU allocation.

I faced several challenges, including the proper modification of the scheduler to choose the highest-priority process while avoiding deadlocks and inconsistencies, as well as managing ties among processes that share the same priority. Additionally, I encountered compiler errors and warnings during the integration of the priority field and the updating of associated functions. Testing the scheduler in a multi-process environment to verify the anticipated execution order necessitated meticulous observation and debugging.

Task 3. Add a readytime field to struct proc, initialize it correctly, and modify ps to print a process's age.

To determine the age of a process, I incorporated a readytime field into the struct proc, which records the current time each time a process transitions to the RUNNABLE state. In the ps command, the age is computed as uptime() - readytime, indicating the duration for which the process has been in a waiting state. This assignment enhanced my understanding of monitoring process states and assessing waiting time within the scheduler.

```
$ ps
pid      state   size   ppid    name
1         2       0      0       init
2         2       0      0       sh
9         4       0      0       ps
$
```

Task 4. Add aging to your priority based scheduler.

Explain your aging policy. Show results from running your priority-based scheduler with aging.

Your tests should show the benefits of using aging.

Answer the questions asked in the pdf.

I learned how to improve a priority-

based scheduler by adding an age strategy to avoid process starvation by finishing Task 4.

In order to enable lower-

priority processes to eventually run, aging necessitated dynamically raising the priority of activities that had waited too long in the ready queue.

This assignment made it easier for me to comprehend how CPU scheduling strikes a compromise between efficiency and fairness, as well as how aging makes sure that all programs get CPU time without being perpetually delayed by higher-priority ones.

Describe any difficulties you ran into with this task and if/how you overcame them.

Extra Credit Task (10 points).

Explain how you implemented the user program (modified pexec) to calculate scheduling metrics, i.e., turnaround time and response time for your priority scheduling with aging policy.